

Infix-Postfix Documentation

Program Design Discussion

This program is designed using a modular, procedural approach with stack-based and tree-based data structures to separate responsibilities and improve readability, maintainability, and correctness. The program allows the user to input either an infix or postfix expression, then performs conversion, evaluation, and tree construction based on user-selected options. The main control flow is handled by the `runProgram` function, which manages the two-level menu system, user interaction, input validation, and repeated execution until termination. All major operations such as validation, conversion, evaluation, tree construction, and traversal are handled through well-defined helper functions. The main function remains minimal and simply calls `runProgram` to begin execution. Input validation is handled carefully to ensure robustness. Expressions must follow the defined grammar, including valid operands, operators, and balanced parentheses for infix input. Invalid inputs are rejected and the user is repeatedly prompted until a correct format is provided. Parsing logic ensures that operands and operators are interpreted correctly while handling optional whitespace and enforcing structural correctness.

Data Structure Design Discussion

Stack and Expression Tree Data Structures

The program uses two primary data structures: a stack implemented using a linked list and an expression tree. The stack is used during infix to postfix conversion and postfix evaluation, while the expression tree is used to represent the hierarchical structure of expressions. Each stack node contains:

- `int` value or `char` data: the stored operand or operator
- `Node*` `next`: pointer to the next element in the stack

The expression tree is implemented using a binary tree structure where each node contains:

- `char` data: the operand or operator
- `TreeNode*` `left`: pointer to the left child
- `TreeNode*` `right`: pointer to the right child

The use of dynamic memory allows both structures to grow as needed without fixed size limitations.

Stack Operation and Management

The stack is managed through helper functions such as `push`, `pop`, and `peek`. These functions ensure correct LIFO behavior and are used consistently throughout conversion and evaluation procedures. The stack is also used to temporarily store operators during infix to postfix conversion, ensuring that precedence and associativity rules are correctly applied. Proper memory management is enforced by clearing all dynamically allocated nodes after use, preventing memory leaks.

Expression Conversion and Evaluation Structure

The program supports two primary computational operations: infix to postfix conversion and postfix evaluation.

- Infix to postfix conversion is performed by scanning the expression from left to right, outputting operands directly and using a stack to manage operators based on precedence and associativity rules. Parentheses are handled by creating and resolving operator regions.

- Postfix evaluation is performed by scanning the expression from left to right and applying operators to the two most recent operands using a stack. Results are pushed back onto the stack until a final value is obtained.

Evaluation supports base 36 operands, where digits and uppercase letters represent values from 0 to 35. The final result is displayed in both base 10 and base 36 formats.

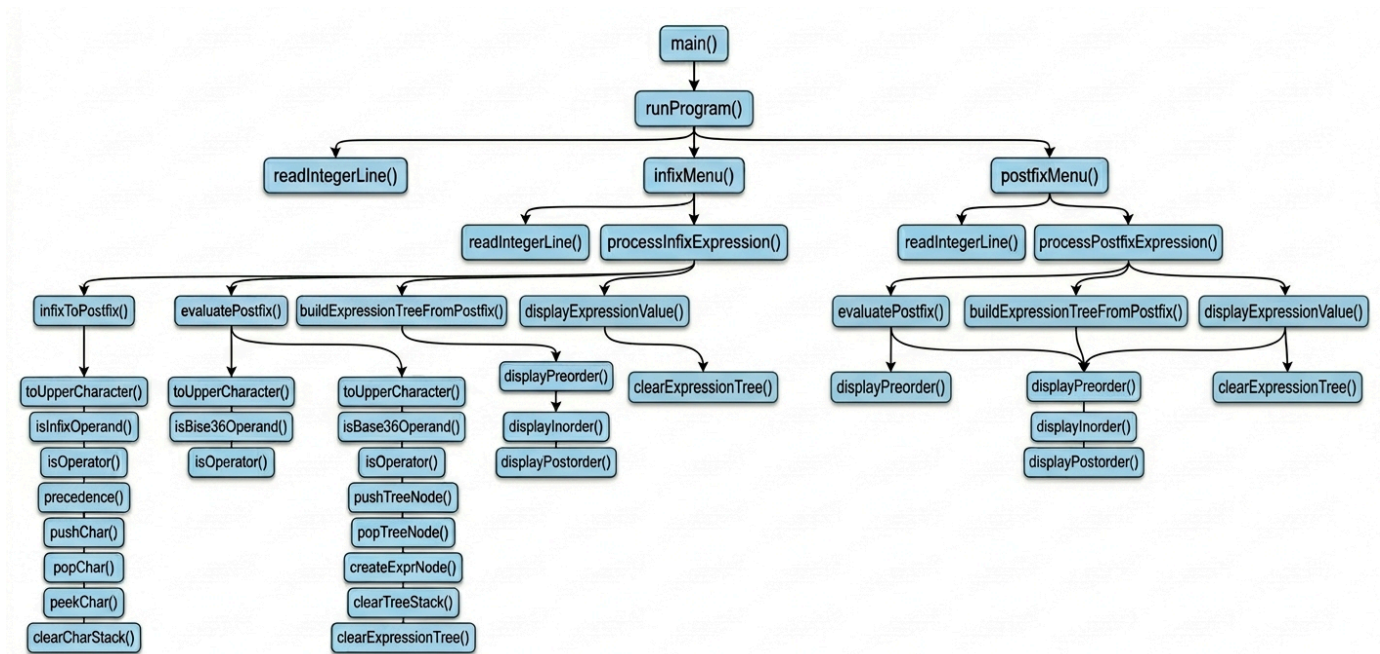
Program Control Structure

The overall program flow is controlled by the runProgram function, which contains:

- int choice: primary menu selection
- string expression: user input expression
- supporting variables for intermediate results and structures

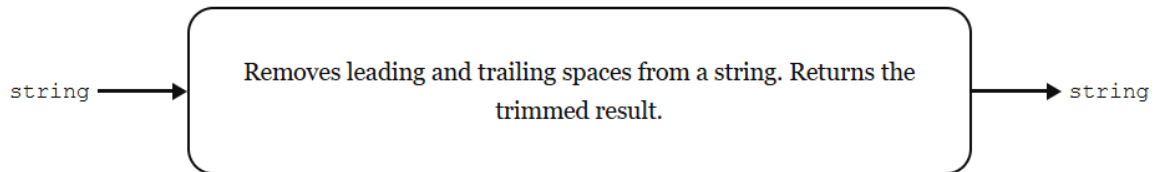
The function implements a two-level menu system. The first level allows the user to choose between infix input, postfix input, or program termination. The second level provides options specific to the chosen input type, such as conversion, evaluation, and tree traversal. After each operation, the program returns to the appropriate menu level, allowing continuous interaction. This centralized control structure ensures that all components of the program work together cohesively while maintaining separation between input, processing, and output logic.

Hierarchical Diagram

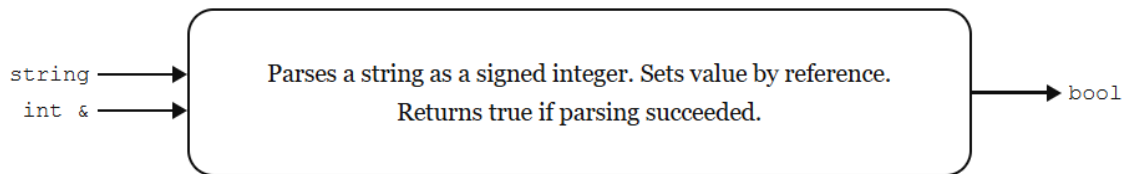


Procedural Specification

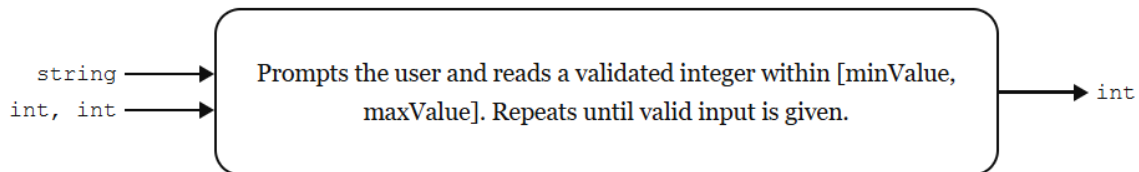
trim



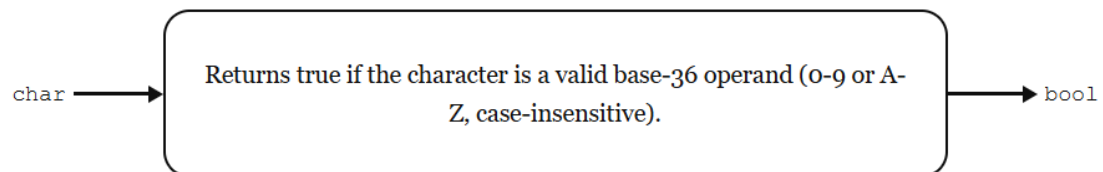
parseInteger



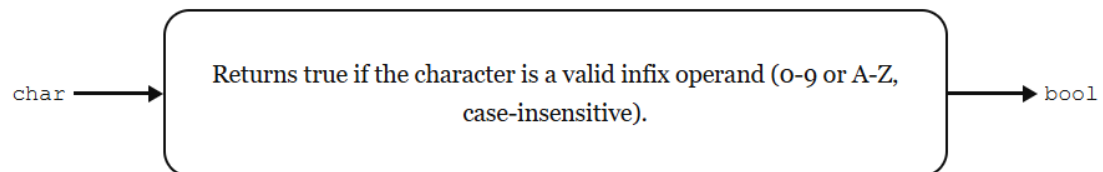
readIntegerLine



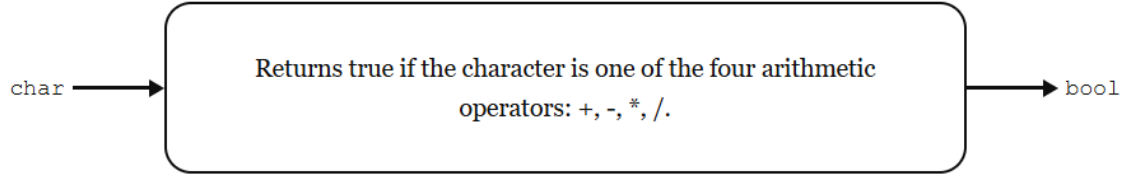
isBase36Operand



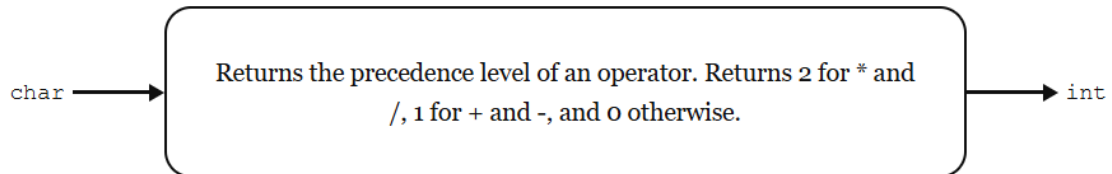
isInfixOperand



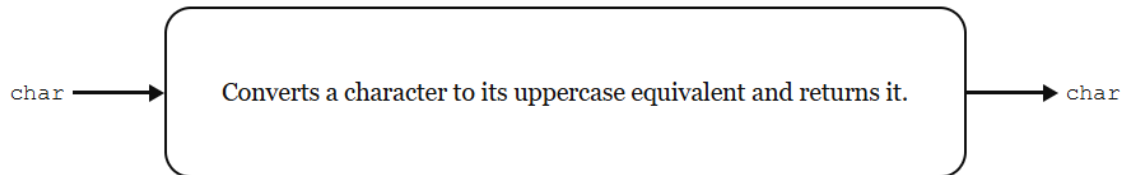
isOperator



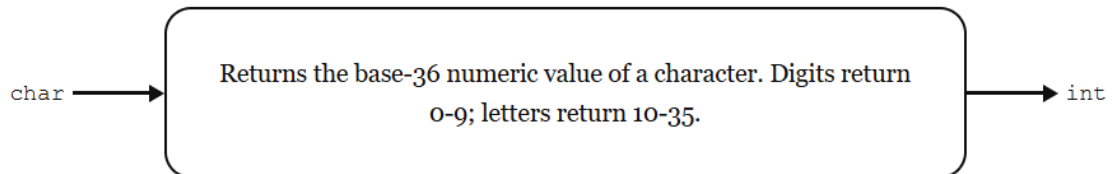
precedence



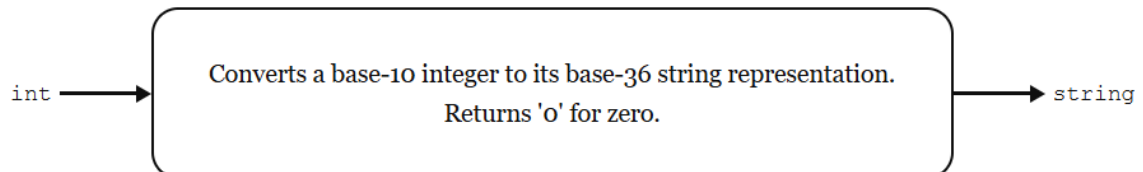
toUpperCharacter



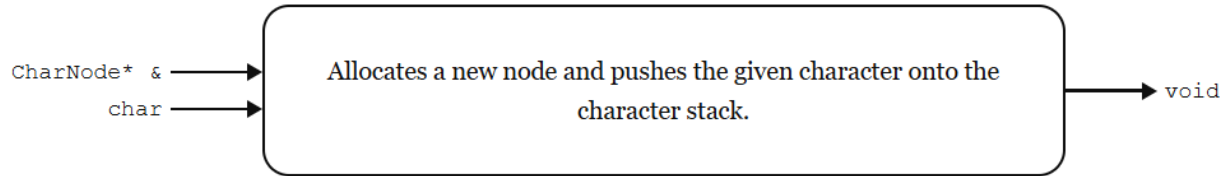
base36Value



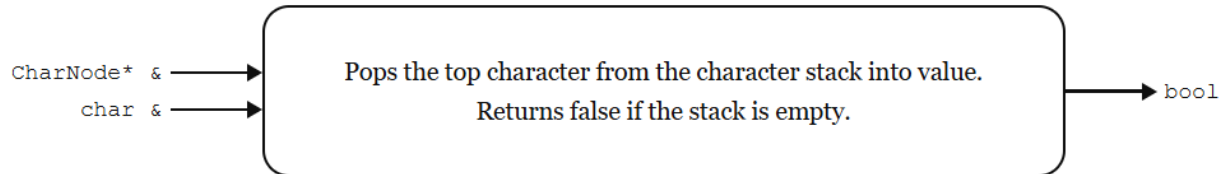
base36String



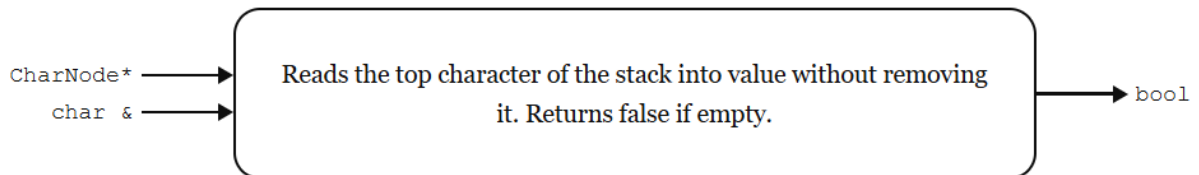
pushChar



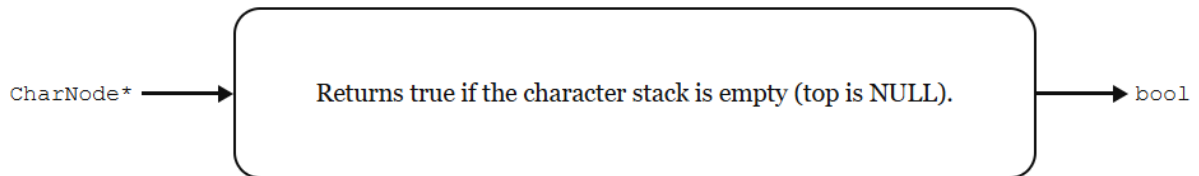
popChar



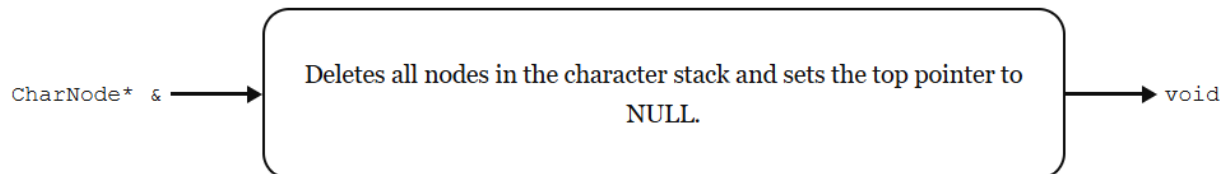
peekChar



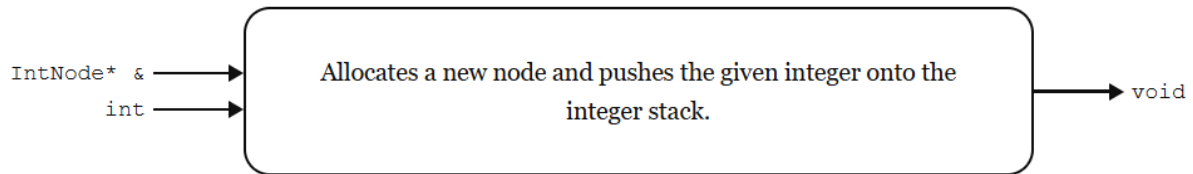
isEmptyCharStack



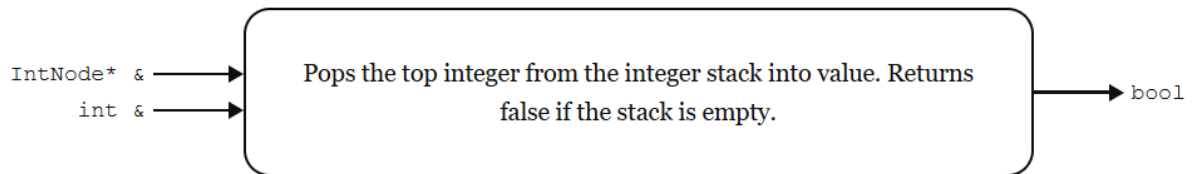
clearCharStack



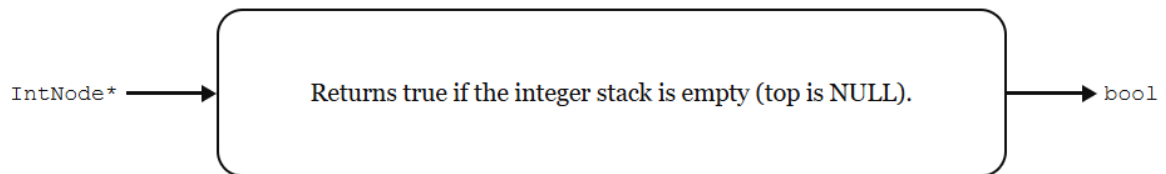
pushInt



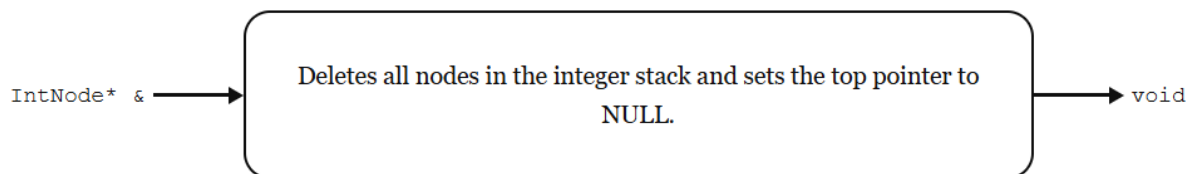
popInt



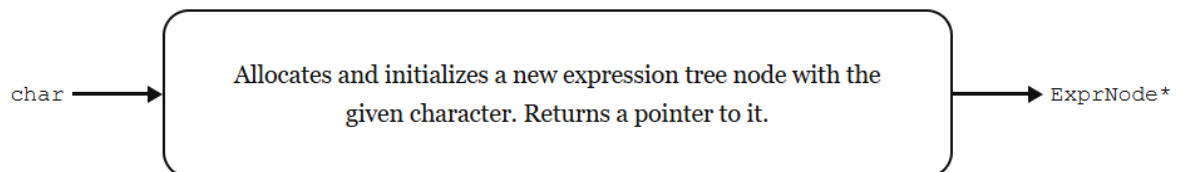
isEmptyIntStack



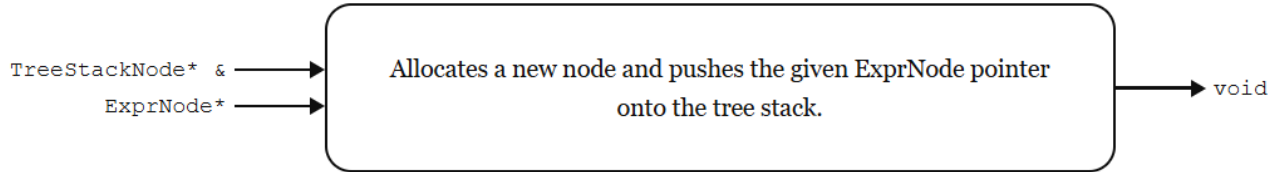
clearIntStack



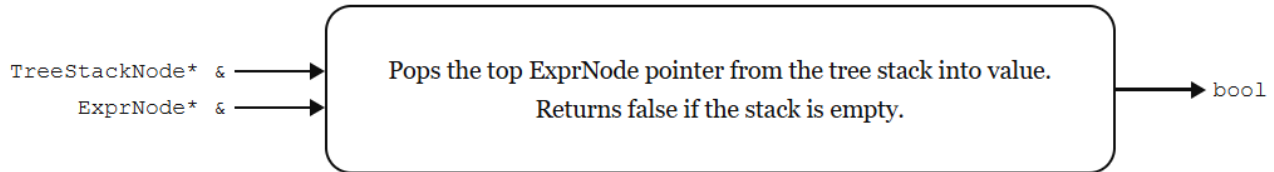
createExprNode



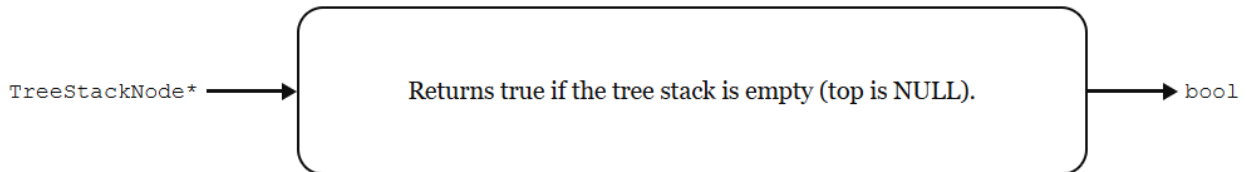
pushTreeNode



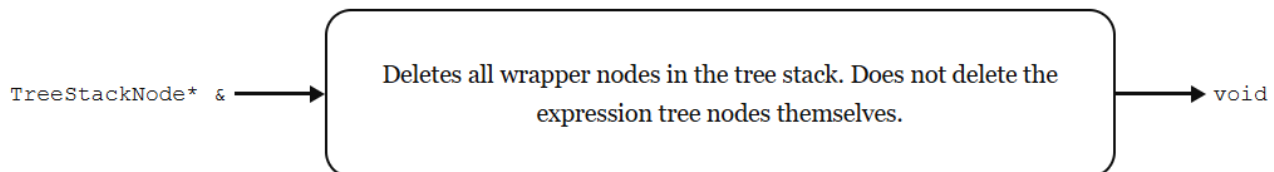
popTreeNode



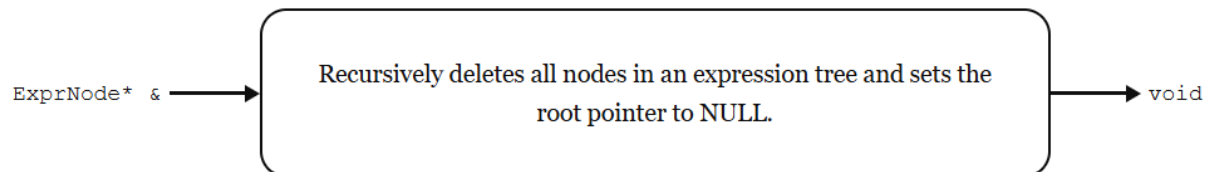
isEmptyTreeStack



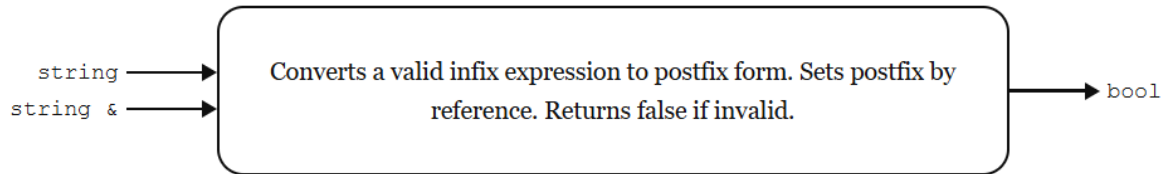
clearTreeStack



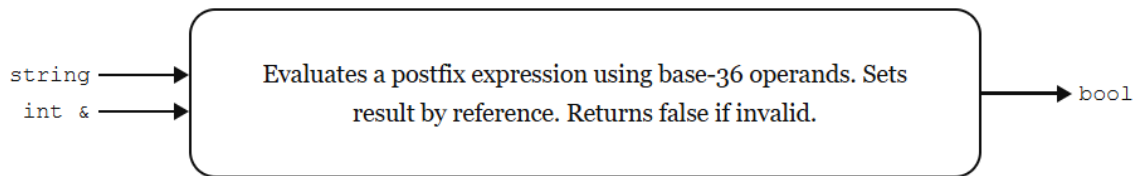
clearExpressionTree



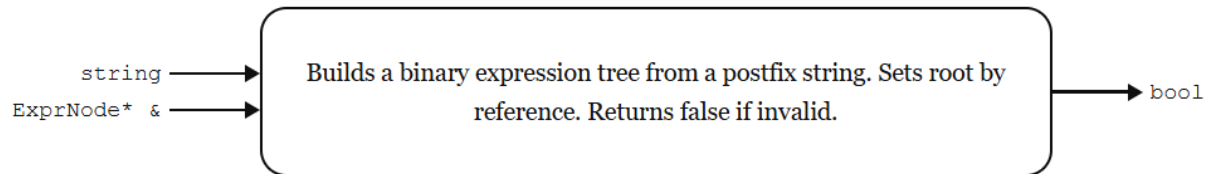
infixToPostfix



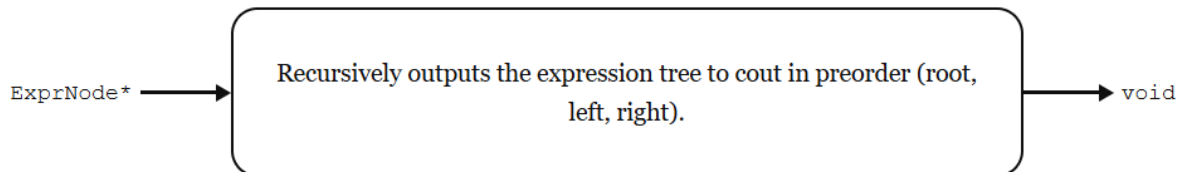
evaluatePostfix



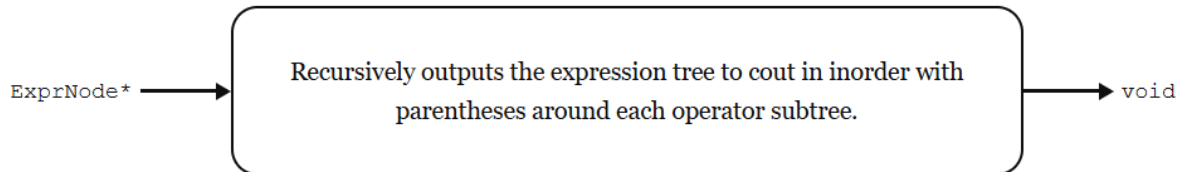
buildExpressionTreeFromPostfix



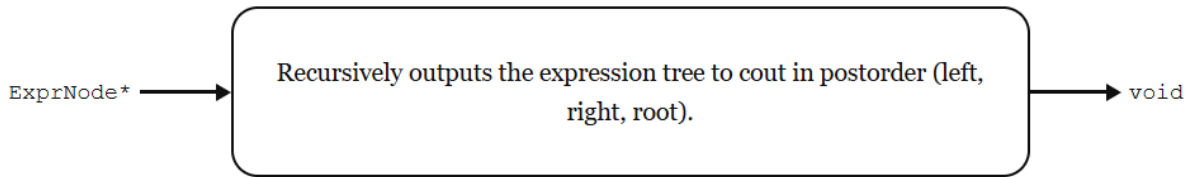
displayPreorder



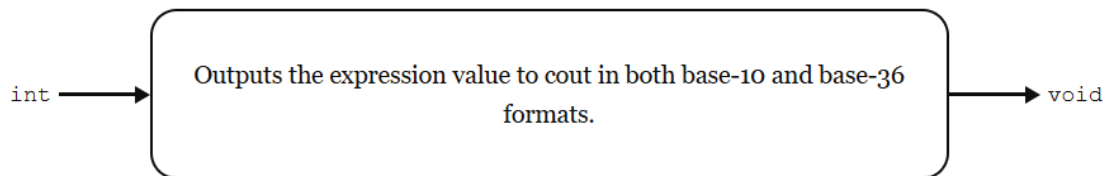
displayInorder



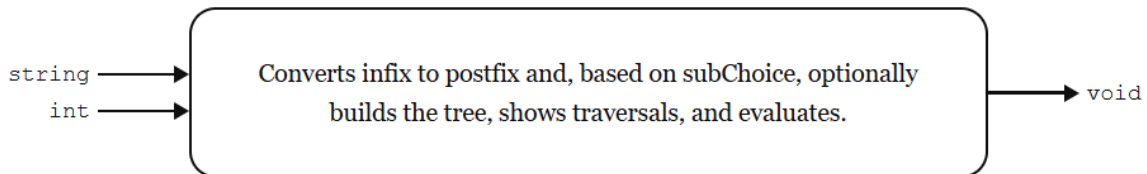
displayPostorder



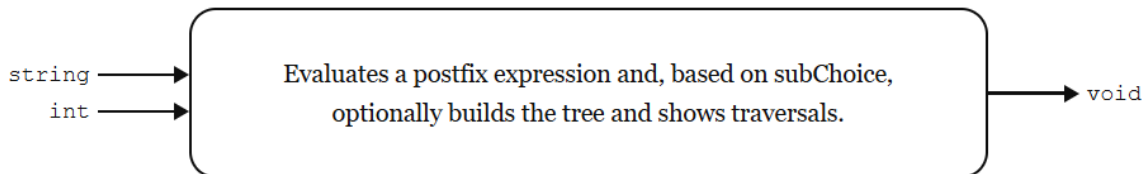
displayExpressionValue



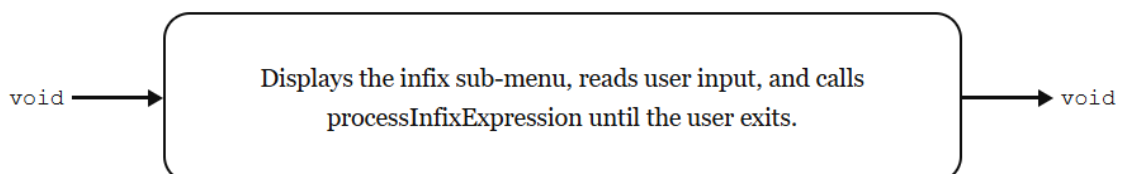
processInfixExpression

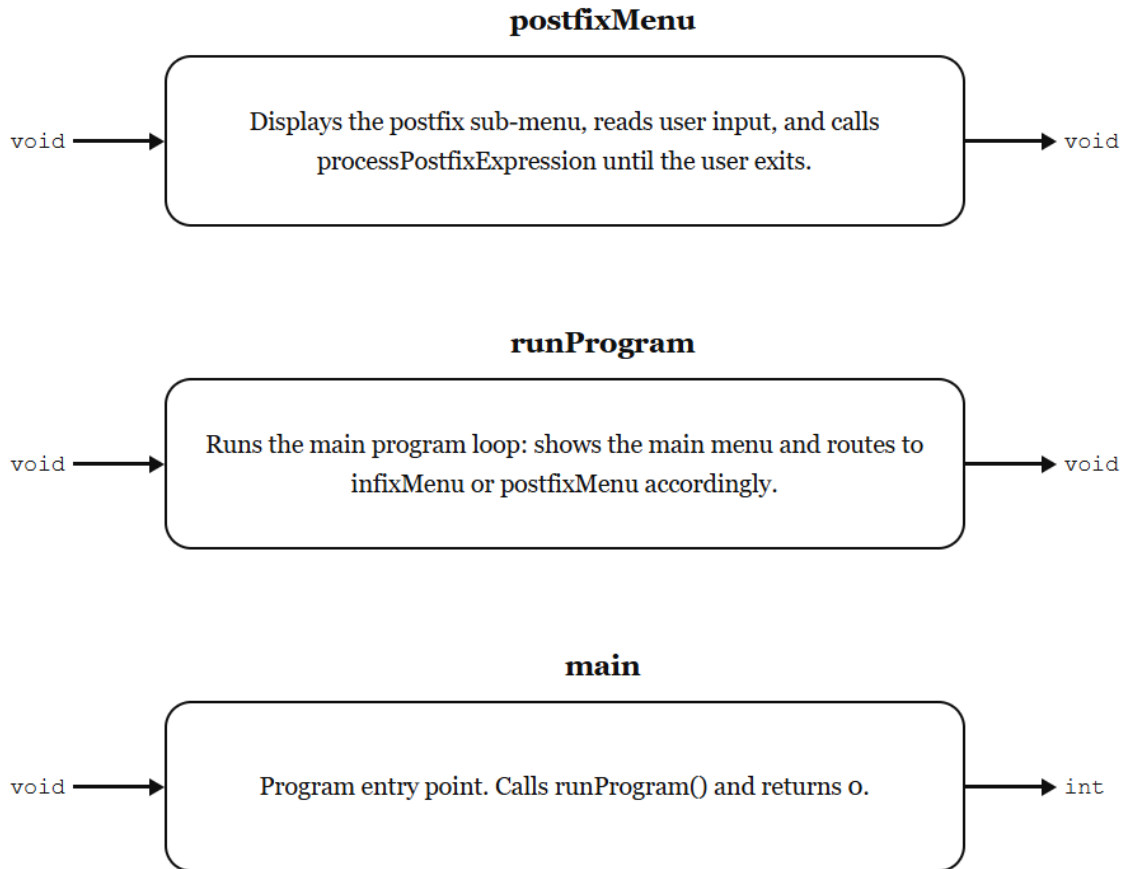


processPostfixExpression



infixMenu





Test Cases with Expected Outputs

Test Case 1 — Exit from Main Menu Immediately

Inputs: main menu choice = 0

Justification: Confirms the program accepts the quit option from the first-level menu and terminates without requesting any expression input.

Expected Output: Program exits cleanly without displaying any postfix expression, tree traversal, or evaluation result.

Test Case 2 — Invalid Main Menu Choice

Inputs: main menu choice = 5, then main menu choice = 0

Justification: Confirms the program rejects an invalid first-level menu option and continues prompting until a valid choice is entered.

Expected Output: Program displays an invalid input message, re-prompts for the main menu, then exits cleanly after 0 is entered.

Test Case 3 — Return from Infix Submenu

Inputs: main menu choice = 1; infix expression = A+B; infix submenu choice = 0; main menu choice = 0

Justification: Confirms the program correctly returns from the infix submenu to the main menu without performing any output operation.

Expected Output: Program accepts the infix expression, returns to the main menu when 0 is selected, and exits cleanly when 0 is entered at the main menu.

Test Case 4 — Return from Postfix Submenu

Inputs: main menu choice = 2; postfix expression = AB+; postfix submenu choice = 0; main menu choice = 0

Justification: Confirms the program correctly returns from the postfix submenu to the main menu without performing traversal or evaluation output.

Expected Output: Program accepts the postfix expression, returns to the main menu when 0 is selected, and exits cleanly when 0 is entered at the main menu.

Test Case 5 — Simple Valid Infix Expression, Postfix Only

Inputs: main menu choice = 1; infix expression = A+B; infix submenu choice = 1

Justification: Confirms the program correctly converts the simplest valid infix expression into postfix form.

Expected Output: Program accepts the expression and displays postfix expression AB+.

Test Case 6 — Simple Valid Infix Expression, Full Output

Inputs: main menu choice = 1; infix expression = A+B; infix submenu choice = 2

Justification: Confirms the program correctly converts infix to postfix, builds the expression tree, prints traversals, and evaluates the result in both base 10 and base 36.

Expected Output: Program displays postfix expression AB+, preorder +AB, inorder (A+B), postorder AB+, and value 21 in base 10 and L in base 36.

Test Case 7 — Infix Expression with Multiplication Precedence

Inputs: main menu choice = 1; infix expression = A+BC; *infix submenu choice = 1*

Justification: Confirms the program correctly applies operator precedence so multiplication is processed before addition.

Expected Output: Program displays postfix expression ABC+.

Test Case 8 — Infix Expression with Parentheses Overriding Precedence

Inputs: main menu choice = 1; infix expression = (A+B)C; *infix submenu choice = 1*

Justification: Confirms the program correctly handles parentheses and changes the order of evaluation.

Expected Output: Program displays postfix expression AB+C.

Test Case 9 — Infix Expression with Nested Parentheses

Inputs: main menu choice = 1; infix expression = A*(B+C/(D-E)); infix submenu choice = 1

Justification: Confirms the program correctly handles nested parentheses and multiple precedence levels.

Expected Output: Program displays postfix expression ABCDE-/+*.

Test Case 10 — Infix Expression with Optional Spaces

Inputs: main menu choice = 1; infix expression = A + B / C - D ; infix submenu choice = 1

Justification: Confirms the program ignores optional whitespace while preserving the correct expression structure.

Expected Output: Program displays postfix expression ABC/+D-.

Test Case 11 — Invalid Infix Expression with Consecutive Operators

Inputs: main menu choice = 1; infix expression = A++B

Justification: Confirms the program rejects infix expressions with improper operator placement.

Expected Output: Program displays an invalid input message and prompts the user to enter another infix expression.

Test Case 12 — Invalid Infix Expression with Missing Right Operand

Inputs: main menu choice = 1; infix expression = A+

Justification: Confirms the program rejects infix expressions that end with an operator.

Expected Output: Program displays an invalid input message and prompts the user to enter another infix expression.

Test Case 13 — Invalid Infix Expression with Missing Left Operand

Inputs: main menu choice = 1; infix expression = *A+B

Justification: Confirms the program rejects infix expressions that begin with a binary operator.

Expected Output: Program displays an invalid input message and prompts the user to enter another infix expression.

Test Case 14 — Invalid Infix Expression with Unbalanced Left Parenthesis

Inputs: main menu choice = 1; infix expression = (A+B

Justification: Confirms the program checks balancing of parentheses and rejects expressions with a missing closing parenthesis.

Expected Output: Program displays an invalid input message and prompts the user to enter another infix expression.

Test Case 15 — Invalid Infix Expression with Unbalanced Right Parenthesis

Inputs: main menu choice = 1; infix expression = A+B)

Justification: Confirms the program rejects expressions containing an unmatched closing parenthesis.

Expected Output: Program displays an invalid input message and prompts the user to enter another infix expression.

Test Case 16 — Invalid Infix Expression with Empty Parentheses

Inputs: main menu choice = 1; infix expression = A+()

Justification: Confirms the program rejects parentheses that do not contain a valid subexpression.

Expected Output: Program displays an invalid input message and prompts the user to enter another infix expression.

Test Case 17 — Simple Valid Postfix Expression, Value Only

Inputs: main menu choice = 2; postfix expression = 12+3*; postfix submenu choice = 1

Justification: Confirms the program correctly evaluates a valid postfix expression and prints only the final value.

Expected Output: Program displays value 9 in base 10 and 9 in base 36.

Test Case 18 — Simple Valid Postfix Expression, Full Output

Inputs: main menu choice = 2; postfix expression = 12+3*; postfix submenu choice = 2

Justification: Confirms the program correctly builds the expression tree from postfix, prints

traversals, and evaluates the result.

Expected Output: Program displays preorder $*+123$, inorder $((1+2)3)$, *postorder* $12+3$, and value 9 in base 10 and 9 in base 36.

Test Case 19 — Postfix Expression Producing Base 36 Letter Output

Inputs: main menu choice = 2; postfix expression = AB*; postfix submenu choice = 1

Justification: Confirms the program correctly interprets alphabetic operands as base 36 digits and converts the final result back to base 36.

Expected Output: Program displays value 110 in base 10 and 32 in base 36.

Test Case 20 — Postfix Expression Producing Negative Result

Inputs: main menu choice = 2; postfix expression = $123*-4+$; postfix submenu choice = 1

Justification: Confirms the program correctly handles subtraction that produces a negative result.

Expected Output: Program displays value -1 in base 10 and -1 in base 36.

Test Case 21 — Postfix Expression with Integer Division

Inputs: main menu choice = 2; postfix expression = $84/$; postfix submenu choice = 1

Justification: Confirms the program uses integer division as required by the assignment.

Expected Output: Program displays value 2 in base 10 and 2 in base 36.

Test Case 22 — Invalid Postfix Expression with Too Few Operands

Inputs: main menu choice = 2; postfix expression = A+

Justification: Confirms the program rejects postfix expressions in which an operator does not have two available operands.

Expected Output: Program displays an invalid input message and prompts the user to enter another postfix expression.

Test Case 23 — Invalid Postfix Expression with Too Many Operands

Inputs: main menu choice = 2; postfix expression = AB

Justification: Confirms the program rejects postfix expressions that do not reduce to exactly one final result.

Expected Output: Program displays an invalid input message and prompts the user to enter another postfix expression.

Test Case 24 — Invalid Postfix Expression with Illegal Character

Inputs: main menu choice = 2; postfix expression = AB\$+

Justification: Confirms the program rejects postfix expressions containing characters outside the allowed operand and operator set.

Expected Output: Program displays an invalid input message and prompts the user to enter another postfix expression.

Test Case 25 — Division by Zero During Evaluation

Inputs: main menu choice = 2; postfix expression = 50/; postfix submenu choice = 1

Justification: Confirms the program detects division by zero during postfix evaluation and prevents an invalid numeric result from being produced.

Expected Output: Program displays an error or invalid evaluation message indicating division by zero and does not print a normal final value.

Results of Test Cases:

```
Main Menu
0. Quit
1. Input an infix expression
2. Input a postfix expression
Enter choice -> 0
PS C:\Users\Brent\OneDrive\Desktop\Data Structures & Algorithms\PostfixInfix>
```

```
Main Menu
0. Quit
1. Input an infix expression
2. Input a postfix expression
Enter choice -> 1

Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> 1

Input infix expression -> A+B

Postfix expression = AB+

Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> 0

Main Menu
0. Quit
1. Input an infix expression
2. Input a postfix expression
Enter choice ->
```

```
Main Menu
0. Quit
1. Input an infix expression
2. Input a postfix expression
Enter choice -> 2

Postfix Menu
0. Return to previous menu
1. Print expression value only
2. Print traversals and expression value
Enter choice -> 1

Input postfix expression -> AB+

Value (base 10) = 21
Value (base 36) = L

Postfix Menu
0. Return to previous menu
1. Print expression value only
2. Print traversals and expression value
Enter choice -> 0

Main Menu
0. Quit
1. Input an infix expression
2. Input a postfix expression
Enter choice -> █
```

```
Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> 2

Input infix expression -> A+B

Postfix expression = AB+
Preorder   = +AB
Inorder    = (A+B)
Postorder  = AB+
Value (base 10) = 21
Value (base 36) = L
```

```
Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> 1

Input infix expression -> (A+B)C

Invalid infix expression.
```

```
Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> 1

Input infix expression -> A*(B+C/(D-E))

Postfix expression = ABCDE-/+*

Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> 1

Input infix expression -> A + B / C - D

Postfix expression = ABC/+D-
```

```
Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> A++B
Invalid input. Please try again.

Enter choice -> 1

Input infix expression -> A++B

Invalid infix expression.

Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> A+
Invalid input. Please try again.

Enter choice -> 1

Input infix expression -> A+

Invalid infix expression.
```

```
Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> 1
```

```
Input infix expression -> *A+B
```

```
Invalid infix expression.
```

```
Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> (A+B
Invalid input. Please try again.
```

```
Enter choice -> 1
```

```
Input infix expression -> (A+B
```

```
Invalid infix expression.
```

```
Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> 1
```

```
Input infix expression -> A+B)
```

```
Invalid infix expression.
```

```
Infix Menu
0. Return to previous menu
1. Print postfix expression only
2. Print postfix expression, traversals, and expression value
Enter choice -> 1
```

```
Input infix expression -> A+()
```

```
Invalid infix expression.
```

Postfix Menu

- 0. Return to previous menu
 - 1. Print expression value only
 - 2. Print traversals and expression value
- Enter choice -> 1

Input postfix expression -> 12+3*

Value (base 10) = 9

Value (base 36) = 9

Postfix Menu

- 0. Return to previous menu
 - 1. Print expression value only
 - 2. Print traversals and expression value
- Enter choice -> 2

Input postfix expression -> 12+3*

Preorder = *+123

Inorder = ((1+2)*3)

Postorder = 12+3*

Value (base 10) = 9

Value (base 36) = 9

Postfix Menu

- 0. Return to previous menu
 - 1. Print expression value only
 - 2. Print traversals and expression value
- Enter choice -> 2

Input postfix expression -> AB*

Preorder = *AB

Inorder = (A*B)

Postorder = AB*

Value (base 10) = 110

Value (base 36) = 32

Postfix Menu

- 0. Return to previous menu
- 1. Print expression value only
- 2. Print traversals and expression value

Enter choice -> 2

Input postfix expression -> 123*-4+

Preorder = +-1*234

Inorder = ((1-(2*3))+4)

Postorder = 123*-4+

Value (base 10) = -1

Value (base 36) = -1

Postfix Menu

- 0. Return to previous menu
- 1. Print expression value only
- 2. Print traversals and expression value

Enter choice -> 1

Input postfix expression -> 84/

Value (base 10) = 2

Value (base 36) = 2

Postfix Menu

- 0. Return to previous menu
- 1. Print expression value only
- 2. Print traversals and expression value

Enter choice -> 2

Input postfix expression -> A+

Invalid postfix expression.

Postfix Menu

- 0. Return to previous menu
- 1. Print expression value only
- 2. Print traversals and expression value

Enter choice -> 1

Input postfix expression -> AB

Invalid postfix expression.

Postfix Menu

0. Return to previous menu

1. Print expression value only

2. Print traversals and expression value

Enter choice -> 1

Input postfix expression -> AB\$+

Invalid postfix expression.

Postfix Menu

0. Return to previous menu

1. Print expression value only

2. Print traversals and expression value

Enter choice -> 1

Input postfix expression -> 50

Invalid postfix expression.